

Modelbildung und Simulation

Lidar-basiertes SLAM mit Kalman-Filter

Verfasst von

Majd Shahrour – Matrikel: 897392

Till Körsmeier – Matrikel: 948657

betreut von

Prof. Dr. Peter Greogorius

Datum

19. März. 2025

Contents

1 Einleitung	5
Teil 	7
2 Verwendete Sensoren	7
2.1 IMU (Inertial Measurement Unit)	7
2.2 Wheel-Encoder	7
3 Kalman-Filter (KF)	8
3.1 Verwendete Matrizen und Vektoren	8
3.2 Funktionsweise des Kalman-Filters	9
3.2.1 Vorhersagephase (Prediction)	10
3.2.2 Korrekturphase (Update)	10
3.3 Eigene Implementierung	11
3.3.1 Zustandsmodell	11
3.3.2 Kernfunktionen des KF-Filters	12
3.3.3 Adaptive Rauschanpassung	13
3.3.4 Integration in ROS2	14
3.4 Auswertung Kalman-Filter	15
3.4.1 Stillstand-Test	17
3.4.2 Gerade Strecke	18
3.4.3 Kurvenfahrt	19
4 Extended Kalman-Filter (EKF)	21
4.1 Eigene Implementierung	21
4.2 Auswertung Extended Kalman-Filter	27
Teil 	29
5 Lokalisierung	29
5.1 Lokale Lokalisierung mit EKF	29
5.2 Globale Lokalisierung	31
5.3 SLAM Simultaneous Localization And Mapping	32
5.3.1 Graph-basierte SLAM	33
5.3.2 Occupancy Grid Map (OGM)	34
Zeitplan	35
Literatur	36

List of Figures

1	Verwendeter Roboter	6
2	Systemarchitektur des Kalman-Filters in ROS2	6
3	Iteration in Kalman-Filter	9
4	Datenfluss des implementierten Kalman-Filters in ROS2	15
5	Vergleich der Geschwindigkeiten von Stillstand-Test	17
6	Vergleich der Geschwindigkeiten von Gerade Stecke Test	18
7	Vergleich der Geschwindigkeiten von Kurvenfahrt Test	19
8	Datenfluss des implementierten Extended-Kalman-Filters in ROS2	26
9	Animierte Darstellung des Störungstests vom Extended Kalman- Filter	27
10	Vergleich der Geschwindigkeiten von dem Störungstest	28
11	TF-Tree des Roboters von der EKF-basierten lokalen Lokalisierung	30
12	Animierte Darstellung der Lösung des kidnapped-robot-Problems mit AMCL	32
13	SLAM-Problem	33
14	Darstellung der Graphenkonstruktion in Graph-SLAM	34

List of Tables

1	Verwendete Matrizen und Vektoren im Kalman-Filter	8
---	---	---

Dieses Dokument besteht aus zwei Hauptteilen:

- Im ersten Teil werden **KF und EKF** im Rahmen des Moduls **Modellbildung & Simulation** untersucht.
- Im zweiten Teil, ab Kapitel fünf, wird die **Lokalisierung** im Rahmen des Moduls **Vertiefung Programmierbare Logik** untersucht

1 Einleitung

Eine der Hauptaufgaben in der Navigation von unbemannten oder autonomen Bodenfahrzeugen (UGVs) ist die Lokalisierung. Der De-facto-Standard hierfür basiert oft auf globalen Navigationssatellitensystemen (GNSS). Real Time Kinematic (RTK) GNSS-Lösungen ermöglichen eine zentimetergenaue Lokalisierung. In vielen Bereichen, wie der Agroförstwirtschaft oder im Indoor-Einsatz, sind satellitengestützte Lösungen jedoch oft nicht verfügbar oder fehleranfällig.

Die präzise Ermittlung der Position und Orientierung eines Roboters ist eine zentrale Herausforderung in der mobilen Robotik, besonders in Umgebungen mit Unsicherheiten und Sensorrauschen. Probabilistische Filtermethoden wie der Kalman-Filter (KF) und der Erweiterte Kalman-Filter (EKF) ermöglichen eine zuverlässige Schätzung der Roboterpose durch Sensorfusion. Dabei werden Informationen aus verschiedenen Quellen wie Odometrie, IMU, LiDAR und GPS kombiniert.

Der Kalman-Filter nutzt ein mathematisches Modell des Roboters, um Vorhersagen zu treffen und diese mit den Sensordaten abzugleichen, wodurch Ungenauigkeiten reduziert werden. Während der klassische KF nur für lineare Systeme geeignet ist, kann der EKF auch nichtlineare Zusammenhänge approximieren und ist daher besonders in der mobilen Robotik weit verbreitet.

Der Kalman-Filter basiert auf dem Bayes-Filter, einem probabilistischen Ansatz zur Zustandsschätzung. Der Bayes-Filter verwendet die bedingte Wahrscheinlichkeit, um die Wahrscheinlichkeit eines Zustands basierend auf vorherigen Zuständen und neuen Messungen zu aktualisieren.

Die Formel des Bayes-Filters lautet:

$$p(x_k | z_k) = \frac{p(z_k | x_k) \cdot p(x_k | z_{k-1})}{p(z_k)} \quad (1)$$

Wobei:

- $p(x_k | z_k)$: Posterior-Verteilung des Zustands x_k gegeben die Messung z_k
- $p(z_k | x_k)$: Likelihood der Messung z_k gegeben den Zustand x_k
- $p(x_k | z_{k-1})$: Prior-Verteilung des Zustands x_k basierend auf den vorherigen Messungen z_{k-1}
- $p(z_k)$: Normalisierungsfaktor (Gesamtwahrscheinlichkeit der Messung z_k)

Diese Formel beschreibt, wie die Wahrscheinlichkeit eines Zustands basierend auf neuen Messungen und vorherigen Zuständen aktualisiert wird, was die Grundlage für den Kalman-Filter bildet.

Diese Arbeit **Lidar-basiertes SLAM mit Kalman-Filter** untersucht im ersten Teil im Rahmen des Module **Modellbildung & Simulation** die Implementierung und Evaluierung des Kalman-Filters. Neben der mathematischen Modellierung

wird die Leistungsfähigkeit dieser Algorithmen anhand realer Sensordaten (IMU und Wheel-Encoder) untersucht. Die Implementierung erfolgt im ROS2 (Robot Operating System 2) auf einem selbst entwickelten mobilen Roboter.

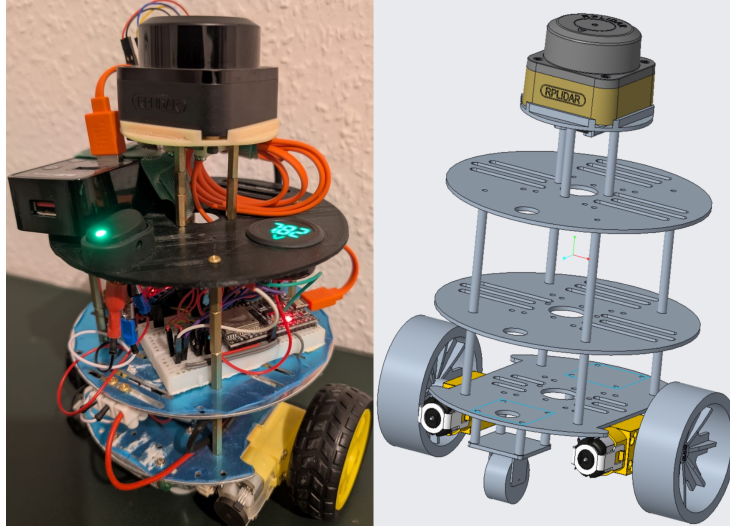


Figure 1: Verwendeter Roboter

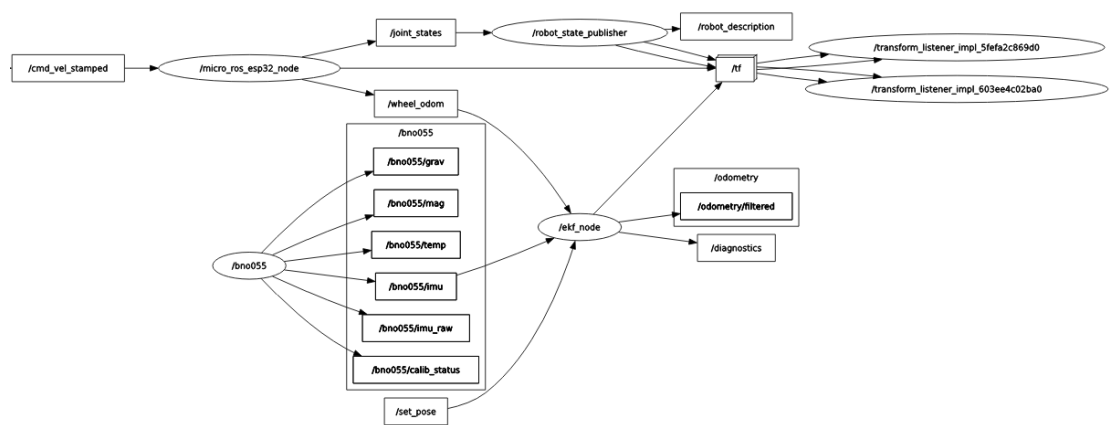


Figure 2: Systemarchitektur des Kalman-Filters in ROS2

Teil |

2 Verwendete Sensoren

In diesem Abschnitt werden die für die Sensorfusion eingesetzten Sensoren zusammenfassend betrachtet. Zum Einsatz kommen Wheel-Encoder als Odometrie-Quelle sowie eine IMU (BNO055), welche neben der Erfassung von Beschleunigungen und Winkelgeschwindigkeiten auch über ein integriertes Magnetometer verfügt, das zur Bestimmung der Ausrichtung (Heading) des Roboters verwendet wird.

Da ROS2 (Robot Operating System 2) als Framework für die Implementierung verwendet wird, spielt die Verfügbarkeit kompatibler ROS2-Treiber eine entscheidende Rolle bei der Auswahl der Sensoren. Die Nutzung von ROS2-Treibern erleichtert die Integration der Sensoren in die ROS2-Umgebung und ermöglicht eine effiziente Kommunikation sowie eine optimierte Datenverarbeitung.

2.1 IMU (Inertial Measurement Unit)

Ein Inertial Measurement Unit (IMU)-Sensor ist eine integrierte Messeinheit, die typischerweise einen Beschleunigungsmesser und ein Gyroskop kombiniert, wodurch die Erfassung von sechs Freiheitsgraden (6-DOF) ermöglicht wird. Konkret misst der Beschleunigungsmesser die linearen Beschleunigungen entlang der drei Raumachsen, während das Gyroskop die Rotationsgeschwindigkeiten um diese Achsen erfasst. Einige IMU-Sensoren beinhalten zusätzlich ein Magnetometer, welches die Orientierung im Erdmagnetfeld bestimmt und somit die Messung auf insgesamt neun Freiheitsgrade (9-DOF) erweitert. Durch die präzise Erfassung des Heading-Winkels eignet sich ein solches 9-DOF-System besonders für anspruchsvolle Lokalisierungsanwendungen.

Aus diesem Grund wird in diesem Projekt der 9-DOF BNO055 von Bosch eingesetzt, um eine möglichst genaue Erfassung der Bewegungs- und Orientierungsdaten sicherzustellen.

(Embedded, n.d.)

2.2 Wheel-Encoder

Der im Projekt eingesetzte Wheel-Encoder basiert auf dem Hall-Effekt und ist in den Getriebemotor SJ01-120 integriert, der mit 6 V betrieben wird und eine Nenndrehzahl von 160 rpm aufweist. Der Sensor erfasst über den Hall-Effekt präzise Impulse, die zur Bestimmung der Rotationsgeschwindigkeit und der zurückgelegten Strecke herangezogen werden.

Der Quadratur-Encoder des Sensors hat eine Auflösung von 8 Impulsen pro Umdrehung der Welle. Nach der Wandlung ergibt dies 960 Impulse pro Umdrehung der Räder. Anhand der gezählten Impulse lässt sich die Umdrehungszahl der Räder exakt ermitteln, was wiederum eine zuverlässige Berechnung der linearen

Geschwindigkeit ermöglicht. Diese präzisen Messwerte sind essenziell für die Lokalisierung und Navigation von Robotersystemen, da sie, insbesondere in Kombination mit weiteren Sensoren wie einer IMU, eine kontinuierliche und robuste Bewegungsschätzung gewährleisten.

3 Kalman-Filter (KF)

Einer der bekanntesten Algorithmen für Datenfusion aus verschiedenen Sensoren sowie fürs Filtern des Rauschens dieser Daten ist der Kalman Filter (KF).

3.1 Verwendete Matrizen und Vektoren

In der folgenden Tabelle werden die Matrizen und Vektoren, die in der Implementierung des Kalman-Filters verwendet werden, zusammengefasst. Diese Tabelle bietet eine Übersicht über die Symbole und deren Bedeutung, um das Verständnis der mathematischen Grundlagen und der Implementierung zu erleichtern.

Table 1: Verwendete Matrizen und Vektoren im Kalman-Filter

Symbol	Beschreibung
$\hat{x}$	Zustandsvektor, enthält die geschätzten Zustände v_x (lineare Geschwindigkeit) und ω_z (Winkelgeschwindigkeit)
P	Kovarianzmatrix, gibt die Unsicherheit der Schätzung an
A	Übergangsmatrix, beschreibt die Systemdynamik
B	Eingabematrix/Steuermatrix, beschreibt den Einfluss der Steuerdaten
H	Messmatrix, beschreibt die Beziehung zwischen den Messungen und den Zuständen
I	Einheitsmatrix, wird zur Berechnung der aktualisierten Kovarianzmatrix verwendet
Q	Prozessrauschkovarianz, modelliert die Unsicherheit der Störungen im System
R	Messrauschkovarianz, modelliert die Unsicherheit der Messungen
u	Steuervektor, enthält die Änderungen der Zustände v_x und ω_z
z	Messvektor, enthält die gemessenen Zustände v_x und ω_z
K	Kalman-Gain, Gewichtungsfaktor zur Aktualisierung der Zustände
y	Innovation, Differenz zwischen der Messung und der Vorhersage

(Przybylski, n.d.)

3.2 Funktionsweise des Kalman-Filters

Der Kalman-Filter (KF) ist ein Algorithmus zur Schätzung von Zuständen eines Systems (z.B. lineare und angulare Geschwindigkeiten) basierend auf unsicheren Messungen. Die Idee ist, aus verrauschten Daten die bestmögliche Schätzung zu berechnen.

Dabei handelt es sich um einen iterativen Prozess, der Zustände eines Systems schätzt. Er kombiniert ein Modell (Vorhersagephase des Systemverhaltens) und Messungen (Korrekturphase), um eine möglichst genaue Schätzung zu liefern, selbst bei Rauschen und Unsicherheiten. KF beinhaltet zwei verschiedene Gleichungssätze, die in jedem k -ten Zustand bzw. jeder Iteration angewendet werden.

Der KF berechnet aufgrund von Steuerdaten u_k , Sensordaten z_k und einer alten Schätzung des Systemzustands $\hat{x}_{k-1|k-1}$ einen neuen verbesserten Schätzwert $\hat{x}_{k|k}$. Die Vorgehensweise ist zeitdiskret, d.h. wir iterieren durch die Vorhersage- und Korrekturphase innerhalb einer vordefinierten Zeit Δt .

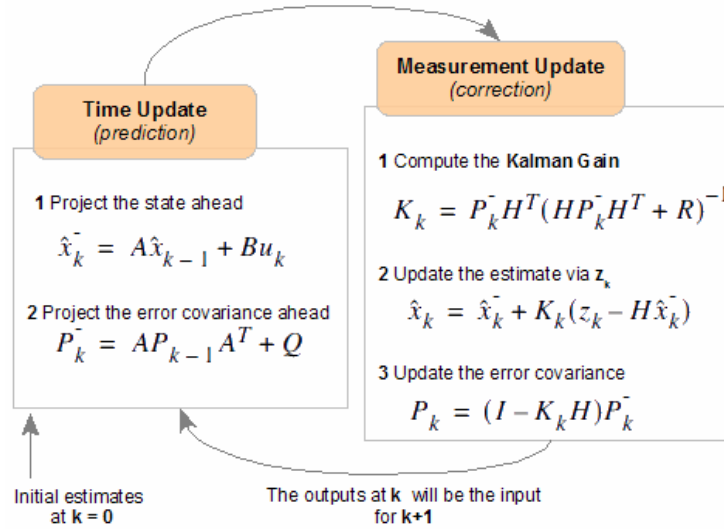


Figure 3: Iteration in Kalman-Filter

In manchen Literaturen wird die Notation $\hat{x}_{k-}$ statt $\hat{x}_{k|k-1}$ verwendet, um die Vorhersage des Zustands zum Zeitpunkt k basierend auf Informationen bis zum Zeitpunkt $k-1$ zu kennzeichnen. Die Notation $\hat{x}_k$ ohne den Strich bedeutet $\hat{x}_{k|k}$, also die aktualisierte Schätzung zum Zeitpunkt k . Das Dach (Caret) $\hat{x}$ weist darauf hin, dass es sich um eine Schätzung handelt.

3.2.1 Vorhersagephase (Prediction)

Schätzen den nächsten Zustand $\hat{x}_{k|k-1}$:

$$\hat{x}_{k|k-1} = A\hat{x}_{k-1|k-1} + Bu_k \quad (2)$$

Schätzen die Unsicherheit der Vorhersage $P_{k|k-1}$:

$$P_{k|k-1} = AP_{k-1|k-1}A^T + Q \quad (3)$$

- P_k : Kovarianzmatrix, gibt an, wie sicher die Vorhersage des neuen Zustands ist.
- A^T : Transponierte von A , wird benötigt, weil eine Kovarianzmatrix symmetrisch ist.
- Q : Prozessrauschkovarianz, modelliert die Unsicherheit der Störungen, die im System selbst auftreten.

Die Vorhersagephase ermöglicht es, von $\hat{x}_{k-1|k-1}$ zu $\hat{x}_{k|k-1}$ zu gelangen, indem nur das Modell genutzt wird.

3.2.2 Korrekturphase (Update)

Berechnung des Kalman-Gain K_k :

$$K_k = P_{k|k-1}H^T(H P_{k|k-1}H^T + R)^{-1} \quad (4)$$

Zustand aktualisieren:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - H\hat{x}_{k|k-1}) \quad (5)$$

$z_k - H\hat{x}_{k|k-1}$ Wird auch als Innovation y_k des KFs bezeichnet

Unsicherheit aktualisieren:

$$P_{k|k} = (I - K_kH)P_{k|k-1} \quad (6)$$

- I : Einheitsmatrix
- z_k : Messwert
- H : Messmatrix
- R : Messrauschkovarianz

Die Korrekturphase ermöglicht es, von $\hat{x}_{k|k-1}$ zu $\hat{x}_{k|k}$ zu gelangen, indem die Messung z_k und der Kalman-Gain K_k berücksichtigt werden.

Der Ausdruck $k|k$ bedeutet, dass das Schätzen des Zustandes basierend auf der Messgleichung z_k erfolgt. Messgleichung:

$$z_k = H_k x_k + v_k \quad (7)$$

- v_k : Messrauschen

3.3 Eigene Implementierung

Die praktische Umsetzung des Kalman-Filters erfolgt in Python als ROS2-Node. Der Filter kombiniert Daten aus der Radodometrie und der IMU, um eine verbesserte Schätzung der Robotergeschwindigkeit zu erhalten. Besonderes Augenmerk wurde auf die dynamische Anpassung der Rauschparameter gelegt, um die Filtereigenschaften an veränderliche Bedingungen anzupassen.

Die Implementierung empfängt Daten über die ROS2-Topics `/wheel_odom` und `/bno055/imu` und veröffentlicht die gefilterten Ergebnisse auf dem Topic `/odometry/filtered`. Zudem wird die Innovation (Differenz zwischen Messung und Vorhersage) auf die Topics `/kf_innovation/linear` und `/kf_innovation/angular` veröffentlicht, um die Filterleistung zu überwachen.

3.3.1 Zustandsmodell

Der implementierte Kalman-Filter verwendet einen zweidimensionalen Zustandsvektor:

$$\hat{x} = \begin{bmatrix} v_x \\ \omega_z \end{bmatrix} \quad (8)$$

Wobei:

- v_x : Lineare Geschwindigkeit des Roboters in x-Richtung
- ω_z : Winkelgeschwindigkeit des Roboters um die z-Achse

Die Wahl dieses Zustandsmodells erlaubt eine direkte Fusion von Odometriedaten (lineare Geschwindigkeit) und IMU-Daten (Winkelgeschwindigkeit). Das Bewegungsmodell ist einfach gehalten, da die Änderung der Geschwindigkeiten direkt als Eingabe verwendet wird:

$$\hat{x}_{k|k-1} = A\hat{x}_{k-1|k-1} + Bu_k \quad (9)$$

Wobei:

- u_k die Differenz der Geschwindigkeiten zwischen aufeinanderfolgenden Odometriedaten darstellt:

$$u_k = \begin{bmatrix} v_{x,k} - v_{x,k-1} \\ \omega_{z,k} - \omega_{z,k-1} \end{bmatrix} \quad (10)$$

In dieser Implementierung sind sowohl die Übergangsmatrix A als auch die Eingabematrix B Einheitsmatrizen.

3.3.2 Kernfunktionen des KF-Filters

Die Implementierung folgt dem klassischen bereits erklärten zweistufigen Ansatz des Kalman-Filters:

Vorhersagephase (Prediction)

```
def statePrediction(self):
    # Predict the next state and update uncertainty

    # \hat{x}_{k/k-1} = A \hat{x}_{k-1/k-1} + B u_{k-1/k-1}
    self.mean_ = (self.A @ self.mean_) + (self.B @ self.motion_)

    # P_{k/k-1} = A P_{k-1/k-1} A^T + Q
    self.variance_ = (self.A @ self.variance_ @ self.A.T) + self.motion_variance_

    self.updateProcessNoise(self.motion_)
```

Korrekturphase (Update)

```
def measurementUpdate(self):
    # Measurement value (z_k) als 2x1 Matrix
    z = np.array([[self.odom_linear_x_], [self.imu_angular_z_]])
    # y_k = z_k - H \hat{x}_{k/k-1}
    innovation = z - self.H @ self.mean_
    # Linear velocity innovation
    self.linear_innovation_pub_.publish(Float32(data=innovation[0][0]))
    # Angular velocity innovation
    self.angular_innovation_pub_.publish(Float32(data=innovation[1][0]))

    # Compute Kalman Gain
    # Vorhersage der Messunsicherheit S_k = H * P_{k/k-1} * H^T + R
    S = self.H @ self.variance_ @ self.H.T + self.measurement_variance_

    # K_k = P_{k/k-1} * H^T / (H * P_{k/k-1} * H^T + R)
    kalman_gain = self.variance_ @ self.H.T @ np.linalg.inv(S)

    # Update the state estimate using measurement
    # \hat{x}_{k/k} = \hat{x}_{k/k-1} + K_k y_k
    self.mean_ = self.mean_ + kalman_gain @ innovation

    # Update the uncertainty
    # P_{k/k} = (I - K_k H) P_{k/k-1}
    self.variance_ = (self.I - kalman_gain @ self.H) @ self.variance_

    self.updateMeasurementNoise(innovation)
```


3.3.3 Adaptive Rauschanpassung

Eine Besonderheit dieser Implementierung ist die dynamische Anpassung der Rauschkovarianzen. Sowohl das Prozessrauschen Q als auch das Messrauschen R werden kontinuierlich basierend auf den aktuellen Daten angepasst:

```
def updateMeasurementNoise(self, innovation):
    """Dynamische Anpassung der Messrauschkovarianz R"""

    error_squared_vx = innovation[0][0] ** 2
    error_squared_w = innovation[1][0] ** 2

    # Update both variance components
    self.measurement_variance_[0,0] =
    0.9 * self.measurement_variance_[0, 0] + 0.1 * error_squared_vx
    self.measurement_variance_[1,1] =
    0.9 * self.measurement_variance_[1, 1] + 0.1 * error_squared_w
```

Dabei wird ein exponentiell gewichteter gleitender Durchschnitt (EWMA) verwendet, der neueren Fehlern mehr Gewicht gibt (10%) als älteren Werten (90%). Dies ermöglicht eine schnelle Anpassung an veränderliche Sensorbedingungen, ohne zu abrupte Änderungen zu verursachen.

Ähnlich wird auch das Prozessrauschen Q dynamisch angepasst, basierend auf der Größe der Bewegungsänderungen.

```
def updateProcessNoise(self, motion_rate):
    """Dynamische Anpassung der Prozessrauschkovarianz Q"""

    self.motion_variance_[0,0] =
    0.9 * self.motion_variance_[0, 0] + 0.1 * motion_rate[0][0] ** 2
    self.motion_variance_[1,1]=
    0.9 * self.motion_variance_[1,1] + 0.1 * motion_rate[1][0] ** 2
```

(Labbe, n.d.)

(Argueta, n.d.b)

(Argueta, n.d.a)

3.3.4 Integration in ROS2

Die Implementierung ist vollständig in das ROS2-Framework integriert, was eine modulare Struktur sowie eine einfache Anbindung an andere Robotiksysteme, wie beispielsweise die geplante Navigation in SLAM, ermöglicht.

1. Datenfluss:

- **Eingabe:** Odometriedaten (`/wheel_odom`) und IMU-Daten (`/bno055/imu`)
- **Ausgabe:** Gefilterte Odometriedaten (`/odometry/filtered`) und Innovation (`/kf_innovation`)

Das Topic `/kf_innovation` veröffentlicht die Innovation des Kalman-Filters als Float32-Wert. Die Innovation ist definiert als die Differenz zwischen der aktuellen Messung und der vorhergesagten Messung: $y_k = z_k - H\hat{x}_{k|k-1}$.

2. Koordinatentransformation:

Der Filter veröffentlicht zudem die erforderlichen Transformationen (TF) zwischen dem Odometry-Frame (`odom`) und dem Base-Roboter-Frame (`base_footprint`) des Roboters:

```
# broadcast TF transform from odom to base_footprint
t = TransformStamped()
t.header.stamp = self.kalman_odom_.header.stamp
t.header.frame_id = 'odom'
t.child_frame_id = 'base_footprint'

# take position from the odometry message
t.transform.translation.x = self.kalman_odom_.pose.pose.position.x
t.transform.translation.y = self.kalman_odom_.pose.pose.position.y
t.transform.translation.z = 0.0

# Use the updated orientation
t.transform.rotation = self.kalman_odom_.pose.pose.orientation

# broadcast transform
TransformBroadcaster(self).sendTransform(t)
```

Durch diesen Ansatz kann die gefilterte Geschwindigkeit des Roboters später problemlos in die ROS2-Navigation eingebunden werden.

Das folgende Diagramm veranschaulicht die Datenflussstruktur des implementierten Kalman-Filters in ROS2. Es zeigt die Sensorfusion aus Raddrehzahldaten (`wheel_odom`) und IMU-Daten (`bno055/imu`) sowie die Weitergabe der gefilterten Odometriedaten und Innovationswerte an verschiedene ROS-Topics.

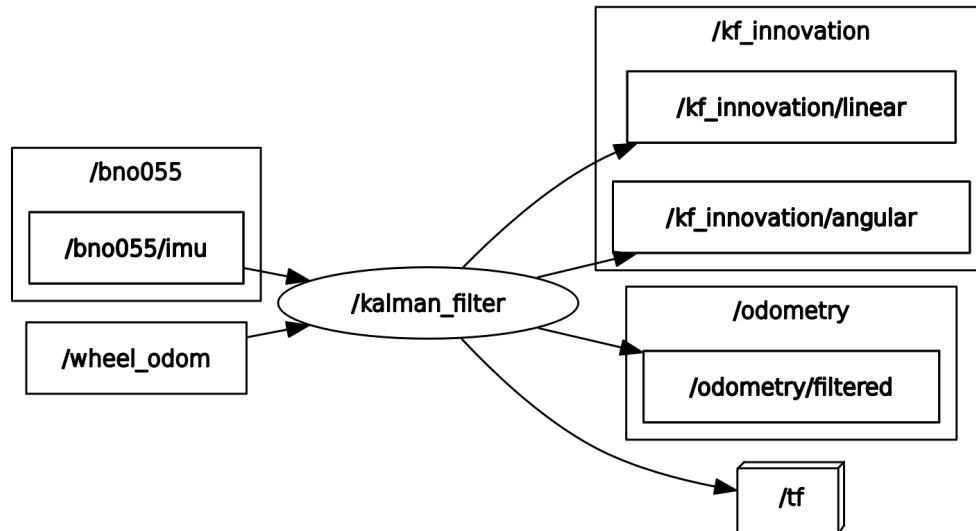


Figure 4: Datenfluss des implementierten Kalman-Filters in ROS2

3.4 Auswertung Kalman-Filter

Für die Auswertung von sowohl KF als auch EKF wird das Debugging-Tool von ROS, **ROS-bags**, verwendet. Ein ROS-Bag ist ein Werkzeug in ROS, das dazu dient, Nachrichten zwischen den verschiedenen Nodes aufzuzeichnen. Diese Nachrichten werden während der Ausführung eines ROS-Systems generiert. Mit ROS-Bags ist es möglich, Daten in Form von Bags aufzuzeichnen und zu speichern. Diese Bags können verschiedene Arten von ROS-Nachrichten enthalten, wie zum Beispiel Sensordaten, Steuerungsbefehle und Odometriedaten.

Der große Vorteil von ROS-Bags besteht darin, dass sie später wieder abgespielt werden können, um die aufgezeichneten Daten zu analysieren, zu visualisieren und für Tests zu verwenden, ohne dass der Roboter erneut ausgeführt werden muss. Dies ermöglicht eine effiziente Überprüfung der Funktionalität eines ROS-Systems und die Identifizierung von Fehlern. Durch das erneute Abspielen der aufgezeichneten Daten können verschiedene Szenarien nachgestellt und die Reaktion des Systems unter diesselben Bedingungen analysiert werden. Für die Evaluierung des Kalman-Filters werden drei Szenarien in Betracht gezogen:

- **Stillstand-Test:** Der Roboter befindet sich im Ruhezustand.
- **Gerade Strecke:** Der Roboter bewegt sich entlang der x-Achse vorwärts und rückwärts.
- **Kurvenfahrt:** Der Roboter dreht sich an Stelle um seine z-Achse.

Für die Evaluierung der Performance des Kalman-Filters wurde ein spezieller ROS2-Node namens `velocity_comparison_node` entwickelt. Dieser Node hat die Aufgabe, die linearen Geschwindigkeiten, die aus den Odometrie-Daten (`/wheel_odom`) stammen sowie die angularen Geschwindigkeiten, die von der IMU (`/bno055/imu`) erfasst werden, mit den durch den Kalman-Filter gefilterten Daten zu vergleichen.

Der `velocity_comparison_node` zeichnet diese Daten auf und erstellt Plots, um die Unterschiede zwischen den verschiedenen Datenquellen zu visualisieren. Dies ermöglicht eine detaillierte Analyse der Filterleistung und hilft dabei, die Genauigkeit und Zuverlässigkeit des Kalman-Filters zu bewerten.

(Robotics, n.d.)

3.4.1 Stillstand-Test

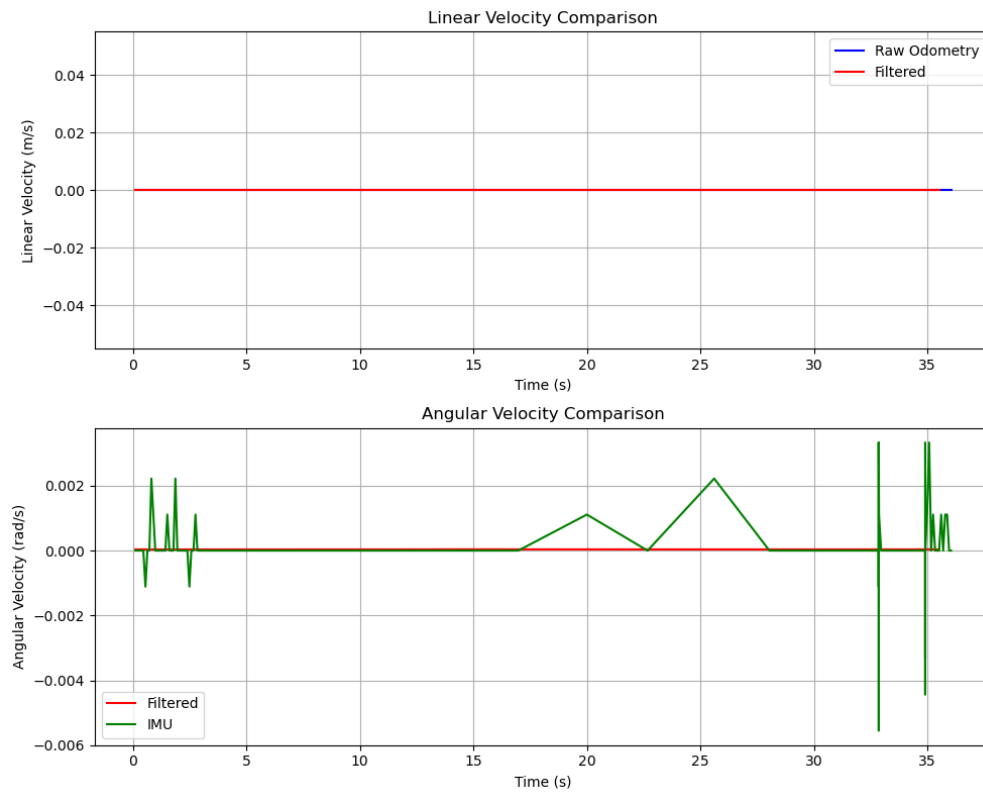


Figure 5: Vergleich der Geschwindigkeiten von Stillstand-Test

Linear Velocity Comparison

Die Plots in Abbildung 5 verdeutlichen, dass sowohl die Roh-Odometrie-Daten als auch die gefilterten Daten eine rauschenfreie, konstante lineare Geschwindigkeit von 0 m/s aufweisen.

Angular Velocity Comparison

Die gefilterten Daten zeigen eine konstante Winkelgeschwindigkeit von 0 rad/s, während die IMU-Daten einige Schwankungen aufweisen, insbesondere zu Beginn und am Ende des Zeitraums, was auf Rauschen zurückzuführen ist.

3.4.2 Gerade Strecke

In diesem Test bewegt sich der Roboter über einen Zeitraum von etwa 40 Sekunden mit konstanter Geschwindigkeit ausschließlich entlang der x-Achse.

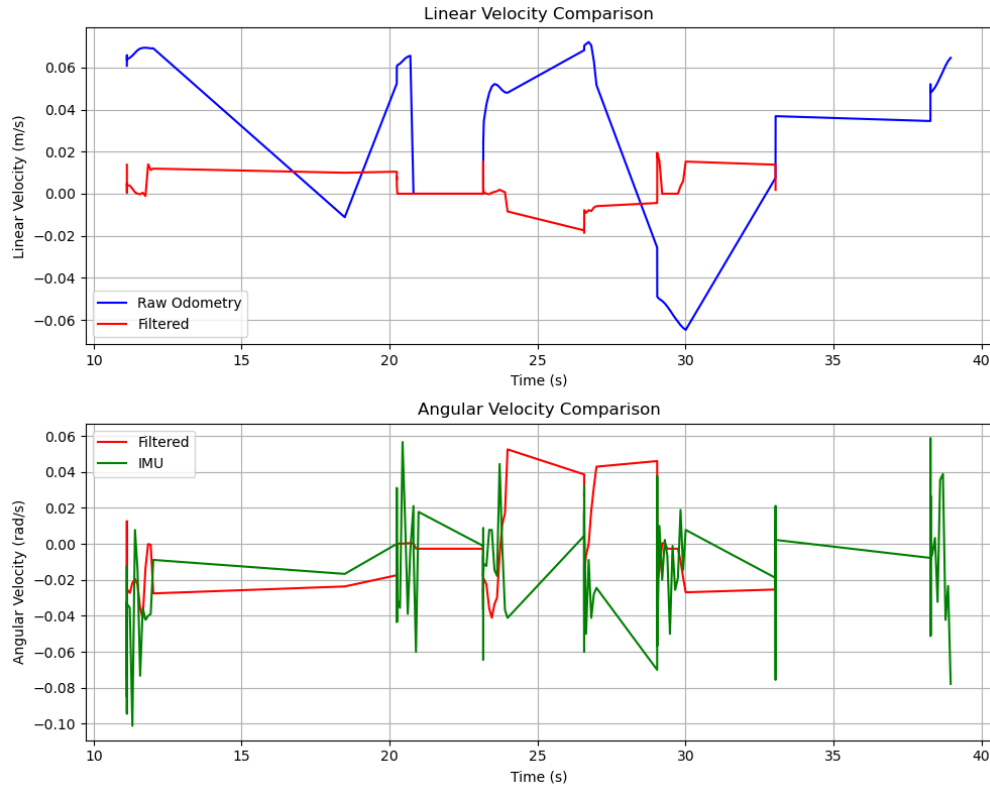


Figure 6: Vergleich der Geschwindigkeiten von Gerade Strecke Test

Linear Velocity Comparison

Die Plots in Abbildung 6 verdeutlichen, dass die Roh-Odometrie-Daten deutliche Schwankungen und Spitzen in der linearen Geschwindigkeit aufweisen, während die gefilterten Daten eine geglättete Version der Geschwindigkeit darstellen, die weniger Schwankungen zeigt. Dies deutet darauf hin, dass der Kalman-Filter erfolgreich das Rauschen in den Roh-Odometrie-Daten reduziert.

Angular Velocity Comparison

Die von Kalman-Filter gefilterten Daten zeigen eine relative konstante Winkelgeschwindigkeit, die nahe bei 0 rad/s liegt, während die IMU-Daten stärkere Schwankungen aufweisen.

3.4.3 Kurvenfahrt

In diesem Test dreht sich der Roboter um seine z-Achse ohne lineare Bewegung über einen Zeitraum von etwa 26 Sekunden. Die Analyse der Geschwindigkeitsverläufe von der Abbildung 7 zeigt deutliche Einschränkungen des Kalman-Filters in nichtlinearen Szenarien wie einer Kurvenfahrt.

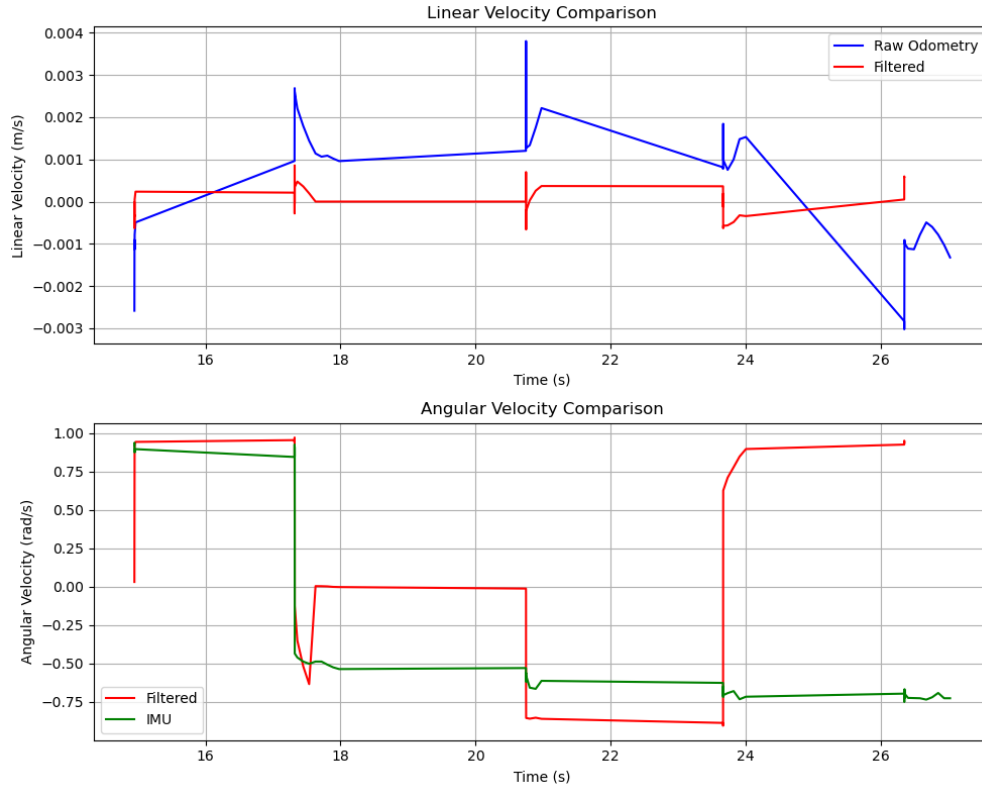


Figure 7: Vergleich der Geschwindigkeiten von Kurvenfahrt Test

Linear Velocity Comparison

Die ungefilterte Roh-Odometrie-Daten zeigen deutliche Schwankungen und Spitzen in der linearen Geschwindigkeit. Die gefilterten Daten zeigen eine geglättete Version der linearen Geschwindigkeit, die weniger Schwankungen aufweist.

Angular Velocity Comparison

Die IMU-Daten zeigen eine relativ konstante Winkelgeschwindigkeit, was erwartet wird, da der Roboter sich hauptsächlich um seine z-Achse dreht, während die gefilterten Daten deutliche Schwankungen in der Winkelegeschwindigkeit zeigen,

die nicht mit den IMU-Daten übereinstimmen.

Bei einer Kurvenfahrt, bei der der Roboter sich um seine z-Achse dreht, treten jedoch nichtlineare Bewegungen auf. In solchen Szenarien kann der lineare Kalman-Filter an Genauigkeit verlieren, da er die Nichtlinearitäten nicht adäquat berücksichtigt. Daher wird in solchen Fällen der erweiterte Kalman-Filter (EKF) eingesetzt, der durch Linearisierung der nichtlinearen Modelle eine bessere Schätzung der Systemzustände ermöglicht.

Der folgende Abschnitt behandelt den Extended Kalman-Filter (EKF) und zeigt, wie dieser Filter die Herausforderungen bei der Schätzung der Position und Orientierung eines mobilen Roboters bewältigt.

4 Extended Kalman-Filter (EKF)

Der klassische Kalman-Filter setzt voraus, dass sowohl das Systemmodell als auch das Messmodell linear sind. Der EKF erweitert den klassischen Kalman-Filter, indem er nichtlineare System- und Messmodelle durch eine Linearisierung in jedem Zeitschritt approximiert. Dies ermöglicht eine genaue Schätzung der Zustände auch bei nichtlinearen Systemen.

Für die Linearisierung der System- und Messmodelle wird die Taylor-Entwicklung erster Ordnung eingesetzt, um das System lokal zu linearisieren.

Lokale Linearisierung Die nichtlinearen Zustands- und Messfunktionen werden um den aktuellen Schätzwert mittels der Taylor-Entwicklung erster Ordnung approximiert.

Jacobi-Matrizen Dabei werden die partiellen Ableitungen der Funktionen (die Jacobi-Matrizen) berechnet, sodass der EKF auch in einem nichtlinearen System zuverlässige Schätzungen liefern kann.

Diese Arbeit wird die Funktionsweise von lokaler Linearisierung und Jacobi-Matrizen aufgrund des Umfangs nicht behandeln.

4.1 Eigene Implementierung

Für die Implementierung des Extended Kalman-Filters (EKF) wird das ROS2-Paket `robot_localization` verwendet. Dieses Paket ist der De-facto-Standard für die Implementierung von EKF in ROS2 und bietet eine robuste und flexible Lösung für die Sensorfusion und Zustandsschätzung.

Das `robot_localization`-Paket unterstützt verschiedene Nachrichtentypen, darunter:

`nav_msgs/Odometry`: Für die Odometrie-Daten `sensor_msgs/Imu`: Für die IMU-Daten `geometry_msgs/PoseWithCovarianceStamped`: Für die Pose-Daten mit Kovarianz `geometry_msgs/TwistWithCovarianceStamped`: Für die Geschwindigkeitsdaten mit Kovarianz

Das `robot_localization`-Paket verfolgt den 15-dimensionalen Zustand des Roboters, der die Position (X, Y, Z), die Orientierung ($roll, pitch, yaw$), die Geschwindigkeit ($\dot{X}, \dot{Y}, \dot{Z}$), die Winkelgeschwindigkeit ($\dot{roll}, \dot{pitch}, \dot{yaw}$) und die Beschleunigung ($\ddot{X}, \ddot{Y}, \ddot{Z}$) umfasst.

Die Implementierung von EKF empfängt, analog zu KF, Daten über die ROS2-Topics `/wheel_odom` und `/bno055/imu` und veröffentlicht die gefilterten Ergebnisse auf dem Topic `/odometry/filtered`.

```

robot_localization = Node(
    package="robot_localization",
    executable="ekf_node",
    name="ekf_node",
    output="screen",
    parameters=[os.path.join(
        get_package_share_directory("my_robot_localization"),
        "config",
        "ekf.yaml"
    )
    ],
)

```

Die Konfiguration des EKF-Nodes erfolgt über eine YAML-Datei, in der die verschiedenen Parameter Einstellungen, wie z.B. das Veröffentlichen von der erforderlichen Transformationen, die Konfiguration von Odometrie- und IMU-Daten.

```

ekf_node:
  ros__parameters:
    frequency: 75.0
    sensor_timeout: 0.1
    two_d_mode: true      #If true, no 3D information in the state estimate.
    transform_time_offset: 0.0
    transform_timeout: 0.0
    print_diagnostics: false
    publish_tf: true
    debug: false
    tf_cache_time: 50.0
    map_frame: map
    odom_frame: odom
    base_link_frame: base_footprint
    world_frame: odom

    # Odom Configuration
    odom0: wheel_odom
    odom0_config: [true, true, false,      # x_pos , y_pos , z_pos,
                  false, false, true,     # roll , pitch , yaw,
                  true, false, false,     # x_vel , y_vel , z_vel,
                  false, false, true,     # roll_vel, pitch_vel, yaw_vel,
                  false, false, false]    # x_accel , y_accel , z_accel]
    odom0_queue_size: 50
    odom0_nodelay: false
    odom0_differential: true
    odom0_relative: false
    odom0_twist_rejection_threshold: 0.5   # Reject unrealistic twists

    # IMU Configuration
    imu0: bno055/imu
    imu0_config: [false, false, false,
                  false, false, true,
                  false, false, false,
                  false, false, true,
                  true, false, false]
    imu0_queue_size: 50
    imu0_nodelay: false
    imu0_differential: false
    imu0_relative: true
    imu0_remove_gravitational_acceleration: true
    imu0_pose_use_child_frame: false
    imu0_linear_acceleration_rejection_threshold: 0.025
    imu0_angular_velocity_rejection_threshold: 0.0
    imu0_orientation_rejection_threshold: 0.0
    imu0_normalize_quaternion: true

```

Wie die Konfiguration zeigt, wird in der Yaml-Datei definiert, welche Daten aus den Odometrie- und IMU-Nachrichten in `wheel_odom` und `/bno055/imu` verwendet werden sollen.

(Moore and Contributors, n.d.)

1. Odometrie-Konfiguration

Die Odometrie-Konfiguration definiert, welche Daten aus den Odometrie-Nachrichten `wheel_odom` verwendet werden sollen. Die `odom0_config`-Liste enthält boolesche Werte, die angeben, ob die entsprechenden Zustandsvariablen in den Filter einbezogen werden sollen.

```
odom0: wheel_odom
odom0_config: [true, true, false, # x_pos, y_pos, z_pos,
               false, false, true, # roll, pitch, yaw,
               true, false, false, # x_vel, y_vel, z_vel,
               false, false, true, # roll_vel, pitch_vel, yaw_vel,
               false, false, false] # x_accel, y_accel, z_accel]
```

- `x_pos` und `y_pos` (true): Die x- und y-Positionen werden verwendet, um die zweidimensionale Position des Roboters zu bestimmen.
- `z_pos` (false): Die z-Position wird nicht verwendet, da der Roboter sich in einer zweidimensionalen Ebene bewegt und die Höhe keine Rolle spielt.
- `roll` und `pitch` (false): Die Roll- und Pitch-Winkel werden nicht verwendet, da der Roboter sich hauptsächlich in der Ebene bewegt und diese Rotationen vernachlässigbar sind.
- `yaw` (true): Der Yaw-Winkel wird verwendet, um die Orientierung des Roboters in der Ebene zu bestimmen.
- `x_vel` (true): Die x-Geschwindigkeit wird verwendet, um die Vorwärtsbewegung des Roboters zu bestimmen.
- `y_vel` und `z_vel` (false): Die y- und z-Geschwindigkeiten werden nicht verwendet, da es sich um einen nicht-holonomen Roboter handelt, der sich nur in der x-Richtung bewegt.
- `roll_vel` und `pitch_vel` (false): Die Roll- und Pitch-Geschwindigkeiten werden nicht verwendet, da der Roboter sich hauptsächlich in der Ebene bewegt und diese Rotationen vernachlässigbar sind.
- `yaw_vel` (true): Die Yaw-Geschwindigkeit wird verwendet, um die Drehgeschwindigkeit des Roboters zu bestimmen.
- `x_accel`, `y_accel` und `z_accel` (false): Die Beschleunigungen werden nicht verwendet, da die Odometrie-Daten diese Informationen nicht zur Verfügung stellen.

2. IMU-Konfiguration

Die IMU-Konfiguration definiert, welche Daten aus den IMU-Nachrichten /bno055/imu verwendet werden sollen. Die imu0_config-Liste enthält boolesche Werte, die angeben, ob die entsprechenden Zustandsvariablen in den Filter einbezogen werden sollen.

```
imu0: bno055/imu
imu0_config: [false, false, false,
              false, false, true,
              false, false, false,
              false, false, true,
              true, false, false]
```

- x_pos, y_pos und z_pos (false): Die Positionsdaten werden nicht verwendet, da die IMU hauptsächlich zur Erfassung von Rotationen und Beschleunigungen dient.
- roll und pitch (false): Die Roll- und Pitch-Winkel werden nicht verwendet, da der Roboter sich hauptsächlich in der Ebene bewegt und diese Rotationen vernachlässigbar sind.
- yaw (true): Der Yaw-Winkel wird verwendet, um die Orientierung des Roboters in der Ebene zu bestimmen. Dies ist wichtig für die Navigation und Bewegungssteuerung.
- x_vel, y_vel und z_vel (false): Die Geschwindigkeitsdaten werden nicht verwendet, da die IMU hauptsächlich zur Erfassung von Rotationen und Beschleunigungen dient.
- roll_vel und pitch_vel (false): Die Roll- und Pitch-Geschwindigkeiten werden nicht verwendet, da der Roboter sich hauptsächlich in der Ebene bewegt und diese Rotationen vernachlässigbar sind.
- yaw_vel (true): Die Yaw-Geschwindigkeit wird verwendet, um die Drehgeschwindigkeit des Roboters zu bestimmen. Diese Information ist wichtig für die Steuerung der Drehbewegungen.
- x_accel (true): Die x-Beschleunigung wird verwendet, um die Vorwärtsbeschleunigung des Roboters zu erfassen. Diese Information kann helfen, plötzliche Änderungen in der Bewegung zu erkennen.
- y_accel und z_accel (false): Die y- und z-Beschleunigungen werden nicht verwendet, da der Roboter sich hauptsächlich in der x-Richtung bewegt.

Das folgende Diagramm veranschaulicht die Datenflussstruktur des implementierten Extended-Kalman-Filters in ROS2. Es zeigt die Sensorfusion aus Rad-drehzahldaten `/wheel-odom` und IMU-Daten (`/bno055/imu`) sowie die Weitergabe der gefilterten Odometriedaten an das (`/odometry/filtered`) ROS-Topic.

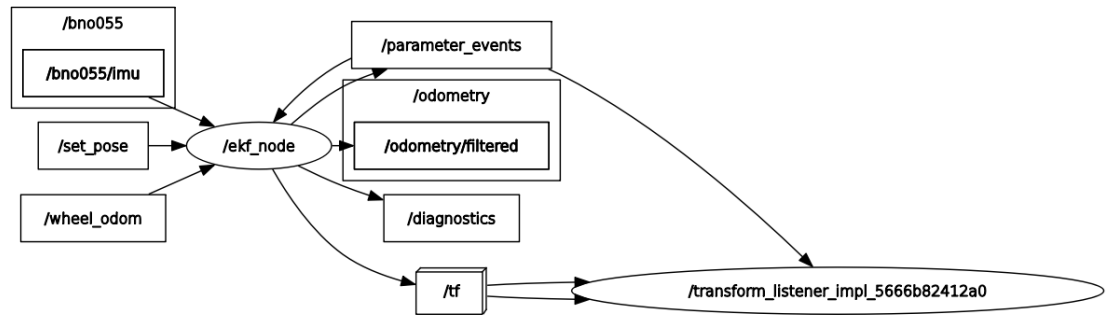


Figure 8: Datenfluss des implementierten Extended-Kalman-Filters in ROS2

4.2 Auswertung Extended Kalman-Filter

Für die Evaluierung der Leistung des Extended Kalman-Filters (EKF) wird mithilfe eines ROS-Bags zur Berücksichtigung des Umfangs nur ein Szenario in Betracht gezogen:

- **Störungstest:** Der Roboter wird während einer Kurvenfahrt angehoben.

RViz ist ein leistungsstarkes Visualisierungstool in ROS2, das zur Darstellung von Sensordaten und Roboterzuständen in Echtzeit verwendet wird. Es ermöglicht die Visualisierung von Daten, die von echten Robotern und Hardware stammen, und bietet eine intuitive Benutzeroberfläche zur Überwachung und Analyse des Systems. Wichtig ist, dass RViz kein Simulator ist, sondern ein Visualisierungstool, das die tatsächlichen Daten und Zustände des Roboters darstellt, um die Interaktion mit der realen Welt zu überwachen und zu analysieren.

Figure 9: Animierte Darstellung des Störungstests vom Extended Kalman-Filter

Im Störungstest wird der Roboter während einer Kurvenfahrt angehoben, wodurch eine Störung in der Bewegung entsteht. Der EKF erkennt diese Störung und nutzt die IMU-Daten, um die unerwünschte Drehbewegung um die z-Achse herauszufiltern, selbst wenn die Räder weiterhin rotieren. Dies verdeutlicht seine Fähigkeit, zwischen der tatsächlichen Bewegung des Roboters und externen Störeinflüssen zu unterscheiden und gleichzeitig präzise Zustandswerte zu liefern. Angesichts der nichtlinearen Charakteristik der IMU-Daten demonstriert der EKF, dass er auch in komplexen Szenarien zuverlässige Schätzungen ermöglichen kann

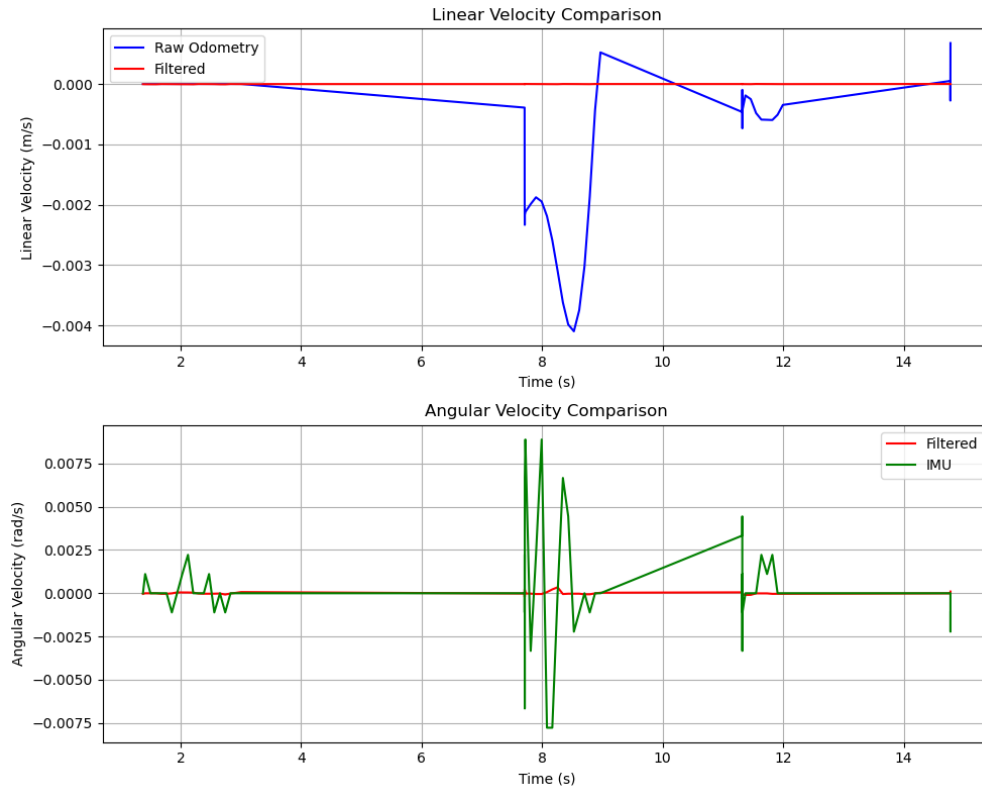


Figure 10: Vergleich der Geschwindigkeiten von dem Störungstest

Die Roh-Odometrie-Daten der linearen Geschwindigkeit spiegeln die Drehung der Räder wider. Diese Drehung, wie in Abbildung 9 gezeigt, findet tatsächlich statt. Die gefilterten Daten hingegen zeigen eine sehr gut geglättete Version der linearen Geschwindigkeit (0 m/s), da sich der Roboter in der Luft befindet und sich nicht bewegt.

Die IMU-Daten der Winkelgeschwindigkeit zeigen vor und nach der Anhebung des Roboters einige Schwankungen. Der Extended Kalman-Filter hingegen zeigt eine konstante Drehgeschwindigkeit von etwa 0 rad/s, da sich der Roboter in der Luft befindet und sich nicht bewegt.

Teil ||

5 Lokalisierung

Die Lokalisierung ist ein zentraler Bestandteil autonomer mobiler Systeme. Sie ermöglicht dem Roboter, die eigene Position und Orientierung (Pose) innerhalb seiner Umgebung zu bestimmen.

1. **Lokale Lokalisierung:**

Die Position des Roboters wird relativ zu seinem Startpunkt bestimmt. Dies erfolgt typischerweise durch die bereits implementierte Fusion von Odometrie- und IMU-Daten mit EKF.

2. **Globale Lokalisierung:**

Sobald eine Karte der Umgebung verfügbar ist, kann der Roboter seine Position relativ zur bekannten Karte bestimmen, unabhängig vom Startpunkt. Dies ermöglicht eine robuste Navigation und Wiedererkennung der eigenen Position.

3. **SLAM (Simultaneous Localization And Mapping):**

Im SLAM wird die Umgebung mittels SLAM erkundet und gleichzeitig eine Karte erstellt. Während der Kartierung lokalisiert sich der Roboter fortlaufend in der erstellten Karte.

5.1 Lokale Lokalisierung mit EKF

Bei der lokalen Lokalisierung wird die Position des Roboters immer relativ zum Startpunkt ermittelt, wodurch der Bezugspunkt ständig der Startpunkt ist, daher wird lokale Lokalisierung auch als relative Lokalisierung bezeichnet. Die Ermittlung der zum Startpunkt relativen Position erfolgt anhand Odometriedaten mithilfe von internen Sensoren, wie z.B. Encoder, wo die Geschwindigkeit bzw. der zurückgelegte Weg ermittelt wird.

Mithilfe von dem bereits implementierten EKF wird die lokale Lokalisierung realisiert.

Die Konfiguration des EKF, wie im Abschnitt '4.1 Eigene Implementierung' gezeigt, ermöglicht die Fusion von Odometrie- und IMU-Daten, um die Roboterpose im lokalen Odometrie-Frame (odom) zu schätzen. Dabei werden die wichtigsten Parameter für die lokale Lokalisierung festgelegt:

```
map_frame: map           # globale Karten-Frame
odom_frame: odom         # Odometrie pose relativ zum Startpunkt
base_link_frame: base_footprint # Beschreibung von Roboterkörper
world_frame: odom        # Zustandsschätzung Referenz-Frame
```

Die Sensordaten und Zustandsinformationen werden in sogenannten **Frames** gespeichert und verarbeitet. Ein Frame definiert einen eindeutigen Bezugspunkt im Raum, beispielsweise die Position des Roboters (**base_footprint**), die lokale

Odometrie (**odom**). Das Frame für die globale Karte (**map**) bleibt in der aktuellen Konfiguration ungenutzt, da keine Karte noch vorhanden ist. Die korrekte Zuordnung der Frames ist entscheidend für die Konsistenz und Genauigkeit der Lokalisierung und Navigation. Fehlerhafte oder inkonsistente Frames führen zu falschen Positionsschätzungen und können die gesamte Navigation des Roboters beeinträchtigen. Daher ist es wichtig, bei der Konfiguration und Implementierung der Lokalisierung auf die korrekte Verwendung und Transformation der Frames zu achten.

Die Abbildung 11 zeigt den Transformationsbaum (TF-Tree) des Roboters, wie er während der lokalen Lokalisierung mit EKF aufgebaut ist. Die einzelnen Frames repräsentieren verschiedene Bezugspunkte des Roboters und seiner Sensoren. Die Hierarchie beginnt beim Odometrie-Frame (**odom**), von dem aus die Transformationen über das Roboter-Frame (**base_footprint** und **base_link**) zu den Sensoren (**laser**, **bno055**, **camera** etc.) weitergegeben werden. Die korrekte Zuordnung und Verkettung der Frames ist entscheidend für die genaue Positionsbestimmung und die zuverlässige Sensorfusion im EKF. Fehlerhafte oder inkonsistente Transformationsketten können zu falschen Lokalisierungsergebnissen führen.

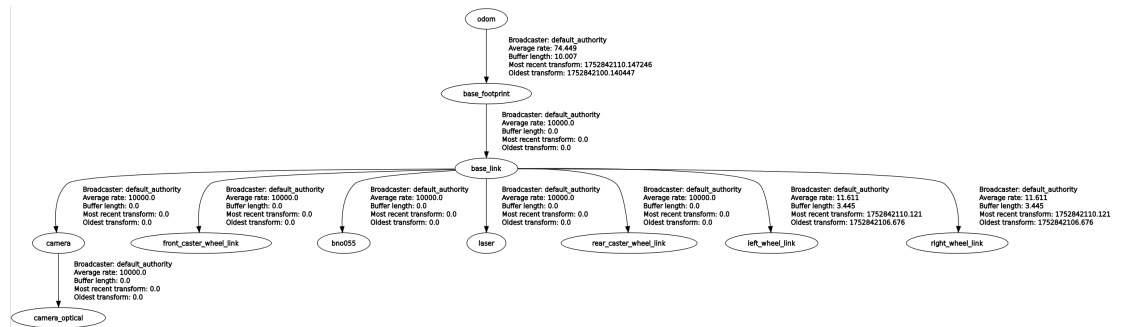


Figure 11: TF-Tree des Roboters von der EKF-basierten lokalen Lokalisierung

5.2 Globale Lokalisierung

Im Gegensatz zur lokalen Lokalisierung, bei der die Startposition des Roboters bekannt ist und die Positionsbestimmung relativ erfolgt, kann bei der globalen Lokalisierung die Startposition unbekannt sein. Die Positionsbestimmung erfolgt absolut, weshalb die globale Lokalisierung auch als absolute Lokalisierung bezeichnet wird. Hierbei nutzt der Roboter externe Sensoren, wie beispielsweise LiDAR, um seine Position unabhängig vom Startpunkt innerhalb einer bekannten Karte zu bestimmen.

Der in dieser Arbeit verwendete Algorithmus für die globale Lokalisierung ist AMCL (Adaptive Monte Carlo Localization), ein weit verbreiteter Ansatz von mobilen Robotern.

AMCL verwendet einen Partikelfilter, um die Position und Orientierung des Roboters innerhalb einer bekannten Karte zu schätzen. Dabei werden viele mögliche Roboterpositionen (partikel) simuliert und mit den aktuellen Sensordaten (in unserem Fall LiDAR-Scans) verglichen. Die wahrscheinlichsten Partikel werden verstärkt, während unwahrscheinliche verworfen werden.

Ein besonderer Vorteil der globalen Lokalisierung ist die Fähigkeit, ein willkürliches Versetzen des Roboters innerhalb der Karte zu erkennen, das als „kidnapped robot“ bekannt ist. Dies wird durch das sogenannte Scan-Matching ermöglicht. Jeder aktuelle LiDAR-Scan wird mit der vorhandenen Karte verglichen, wobei markante Merkmale und Strukturen der Umgebung erkannt und abgeglichen werden. So kann der Roboter auch nach einem beliebigen Versetzen seine Position zuverlässig wiederfinden. LiDAR-basierte globale Lokalisierung ist besonders robust gegenüber Störungen und eignet sich für komplexe, strukturierte Umgebungen.

Die folgende Animation zeigt, wie der AMCL-Algorithmus das „kidnapped robot“-Problem löst. Der Roboter wird innerhalb der Karte willkürlich versetzt, und der Algorithmus erkennt dieses Versetzen und korrigiert die geschätzte Position des Roboters entsprechend.

Die für die globale Lokalisierung notwendige Karte wird zuvor mit Hilfe von SLAM erstellt.

Figure 12: Animierte Darstellung der Lösung des kidnapped-robot-Problems mit AMCL

5.3 SLAM Simultaneous Localization And Mapping

Häufig ist keine Karte der Roboter-Umgebung a priori verfügbar und der Roboter soll eine Karte aus Sensor- und Bewegungsdaten erst erstellen.

Bei SLAM wird die Umgebung erforscht und die eigene Position innerhalb dieser Umgebung ermittelt. es ermöglicht also eine gleichzeitige Kartierung und Lokalisierung.

Es lässt sich zusammenfassen, dass SLAM sich von den zwei unabhängigen Modulen (Lokalisierung und Kartierung) zu einem vollständigen System entwickelt hat, in welchem diese beiden Module verbunden sind und sich gegenseitig unterstützen.

Das führt zu einem zentralen Problem bei SLAM, das **chicken-egg problem**, das als SLAM-Problem bekannt ist:

- Liegt bereits eine präzise Karte der Umgebung vor, ist die Lokalisierung des Roboters darin einfach und zuverlässig möglich.
- Ist hingegen die genaue Position des Roboters bekannt, kann eine Karte der Umgebung problemlos erstellt werden.

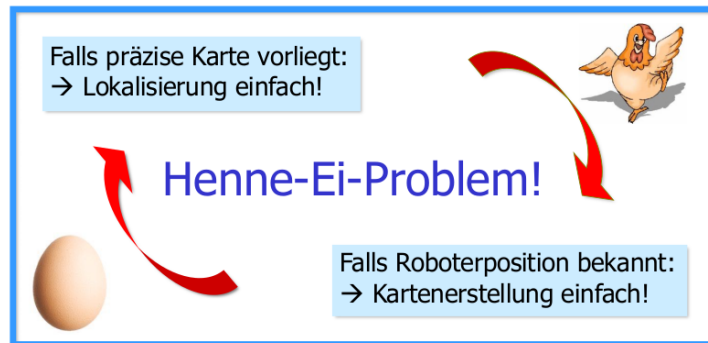


Figure 13: SLAM-Problem

Im praktischen Einsatz sind jedoch meist weder eine exakte Karte noch die genaue Roboterposition zu Beginn verfügbar. SLAM muss daher beide Aufgaben, die Lokalisierung und die Kartenerstellung, gleichzeitig lösen. Dies macht SLAM zu einer besonders anspruchsvollen Herausforderung in der mobilen Robotik. Die Kartierung bzw. das Mapping bezieht sich auf die Fähigkeit von autonomen mobilen Robotern, eine unbekannte Umgebung zu erkunden und daraus eine Karte zu erstellen.

5.3.1 Graph-basierte SLAM

Das in dieser Arbeit verwendete SLAM-Verfahren basiert auf LiDAR-Daten und kombiniert Grid-Mapping mit graph-basierter Optimierung. Dabei werden die Roboterposen x_i zu verschiedenen Zeitpunkten als Knoten in einem Graphen dargestellt. Die Kanten verbinden jeweils aufeinanderfolgende Posen über Bewegungsinformationen (Odometrie) und nicht-aufeinanderfolgende Posen über sogenannte Loop-Closures, wenn der Roboter einen bereits bekannten Bereich erneut besucht.

Bei LiDAR-basiertem Grid-Mapping werden keine expliziten Landmarken m_i als Knoten verwendet. Stattdessen steht die Optimierung der Robotertrajektorie und der daraus resultierenden Karte im Vordergrund.

Die Optimierung des Graphen erfolgt typischerweise über eine spärliche Information-Matrix, da jede Pose nur mit wenigen anderen Posen verbunden ist. Dies ermöglicht eine effiziente und konsistente Schätzung der Robotertrajektorie und der Umgebungskarte, auch in großen und komplexen.

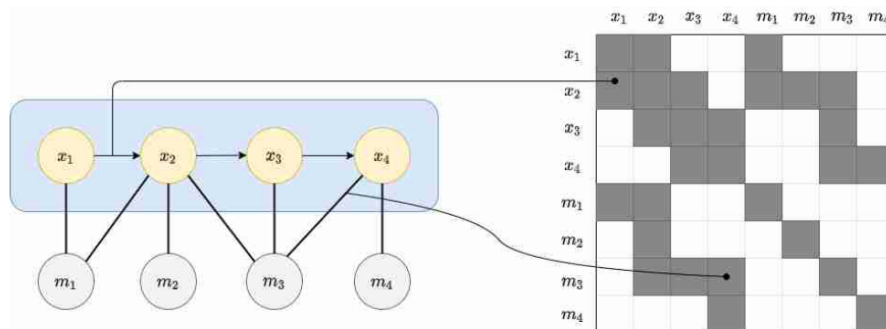


Figure 14: Darstellung der Graphenkonstruktion in Graph-SLAM

5.3.2 Occupancy Grid Map (OGM)

Die Occupancy Grid Map ist das zentrale Kartenmodell in graph-basiertem SLAM. Dabei wird die Umgebung als ein zweidimensionales Raster aus Zellen dargestellt. Jede Zelle enthält einen Wert, der angibt, wie wahrscheinlich es ist, dass sich an dieser Stelle ein Hindernis befindet (belegt), frei ist oder unbekannt bleibt.

Die OGM wird fortlaufend durch die LiDAR-Scans aktualisiert, während sich der Roboter bewegt. So entsteht eine detaillierte und metrisch genaue Karte der Umgebung, die sowohl für die Navigation als auch für die Lokalisierung genutzt werden kann.

Dieses Kartenmodell ist besonders robust gegenüber Sensorrauschen und eignet sich sehr gut für die Echtzeitverarbeitung in mobilen Robotersystemen

Schwellwerte:

Die Schwellenwerte für besetzte und freie Zellen müssen festgelegt werden.

- **occupied_thresh:** Zellen mit einem Wert über 0.65 gelten als „besetzt“ (Hindernis) und werden in der Karte in Schwarz dargestellt.
- **free_thresh:** Zellen mit einem Wert unter 0.25 gelten als „frei“ und werden in der Karte in weiß dargestellt.

occupied_thresh: 0.65

free_thresh: 0.25

Unerforschte Zellen werden innerhalb der Karte in grau dargestellt.

Auflösung:

Die Auflösung der Karte wird über den Parameter **resolution** definiert, in unserem Fall 0.01 für 1 cm pro Zelle. Eine feine Auflösung ermöglicht eine detaillierte Kartendarstellung, erhöht aber den Speicherbedarf und die Rechenlast.

Zeitplan

LIDAR-basierter SLAM mit Kalman-Filter



Literatur

- Argueta, Carlos. n.d.a. “Recursive State Estimation with Kalman Filters and Ros 2.” <https://soulhackerslabs.com/recursive-state-estimation-with-kalman-filters-and-ros-2-b869d3775357>.
- . n.d.b. “Sensor Fusion with the Extended Kalman Filter in Ros 2.” <https://soulhackerslabs.com/sensor-fusion-with-the-extended-kalman-filter-in-ros-2-d33dbab1829d>.
- Embedded, Inventor. n.d. “What Is an Imu Sensor? A Complete Guide for Beginners.” <https://embeddedinventor.com/what-is-an-imu-sensor-a-complete-guide-for-beginners/>.
- Labbe, Roger. n.d. “Kalman and Bayesian Filters in Python.” <https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python>.
- Moore, Tom, and Contributors. n.d. “Robot_localization: A Package for State Estimation and Sensor Fusion in Ros.” https://github.com/cra-ros-pkg/robot_localization.
- Przybylski, Sasha. n.d. “The Math Behind Extended Kalman Filtering.” https://medium.com/@sasha_przybylski/the-math-behind-extended-kalman-filtering-0df981a87453.
- Robotics, Open. n.d. “Recording and Playing Back Data.” <https://docs.ros.org/en/foxy/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html>.